



Sistema de Recomendação de Produtos

TCC — Ciência da Computação · Quasar Barber Distribuidora

Node.js 18+

React 18

Supabase

Redis

Association Rules

Autor: Felipe Pereira | **Data:** Maio de 2026 | **Prazo:** 17 de Agosto de 2026

Índice

1. Objetivo do Projeto
2. Contexto de Negócio
3. Arquitetura do Sistema
4. Stack Tecnológico
5. Estrutura do Banco de Dados
6. Algoritmo de Recomendação
7. API REST — Endpoints
8. Cache Redis
9. Testes Unitários
10. Progresso por Semana (Semanas 1–4)
11. Arquivos do Projeto
12. Decisões Técnicas e Armadilhas
13. Próximos Passos (Semanas 5–12)

1. Objetivo do Projeto

Desenvolver um **sistema de recomendação de produtos B2B** para a Quasar Barber, distribuidora de produtos para barbearias em Codó-MA. O sistema analisa o histórico de vendas para identificar padrões de compra e sugerir automaticamente produtos complementares no momento do pedido.

O trabalho aplica o algoritmo de **Association Rules (Market Basket Analysis)** — técnica consolidada em Data Mining — adaptada ao contexto B2B.

Problema resolvido

- Vendedores não sabem quais produtos sugerir em cada pedido
- Oportunidades de cross-sell perdidas por falta de análise de dados
- Decisões baseadas em intuição, não em histórico quantitativo

Solução proposta

- API REST: recebe produto → retorna top 3 produtos comprados junto
- Dashboard React para visualizar padrões e métricas
- Algoritmo treinado com dados reais de vendas da distribuidora

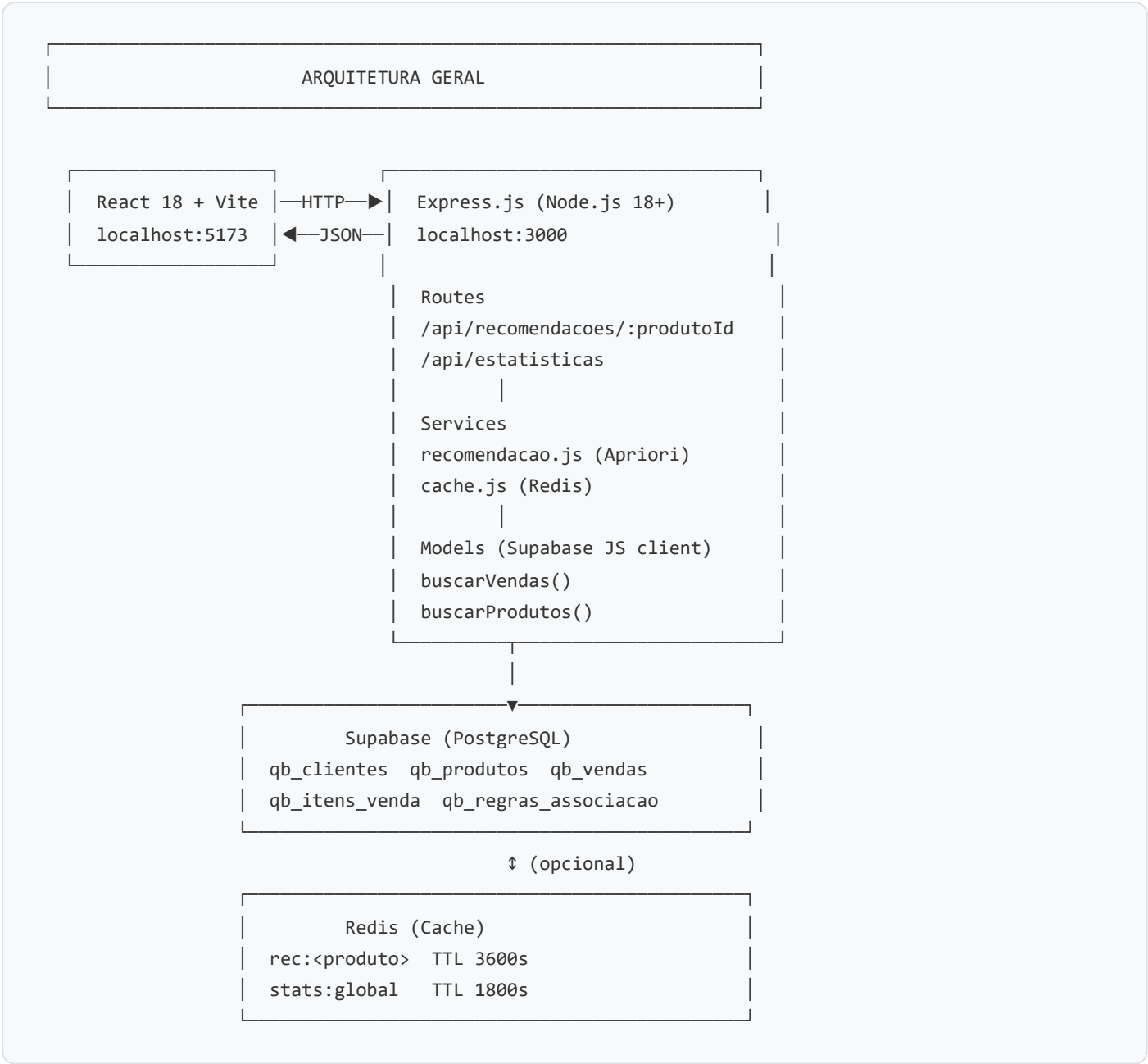
Exemplo: Vendedor informa "Fox For Men Barba" → sistema retorna "Espuma para Barba Premium (43% confiança)" e "Tônico de Barba (43%)" como sugestões.

2. Contexto de Negócio

Item	Detalhe
Empresa	Quasar Barber — Distribuidora de produtos para barbearias
Localização	Codó, Maranhão (MA)
Proprietário	Felipe Pereira
Tipo de venda	B2B: distribuidora → barbearias
Dataset atual	Dados mock realistas (CSV real sem granularidade de itens)
Prazo do TCC	17 de Agosto de 2026
Horas planejadas	12 semanas × 6h = 72 horas

Limitação dos dados reais: O CSV Vendas_2026.csv contém apenas totais por cliente (sem produtos por pedido), sendo inutilizável para Association Rules. Dados mock foram criados para desenvolvimento. Quando disponível o relatório de itens por pedido, os dados reais substituem o mock sem alterar código.

3. Arquitetura do Sistema



4. Stack Tecnológico

Backend

Tecnologia	Versão	Papel
Node.js	18+	Runtime JavaScript server-side
Express.js	4.18	Framework HTTP / roteamento
@supabase/supabase-js	2.38	Cliente PostgREST para banco de dados
redis	5.x	Cliente Redis para cache
dotenv	16.x	Variáveis de ambiente
cors	2.8	Middleware CORS
jest + supertest	29.x / 6.x	Testes unitários e de integração

Frontend

Tecnologia	Versão	Papel
React	18	Biblioteca UI declarativa
Vite	5.x	Bundler e servidor de desenvolvimento
Axios	1.x	Cliente HTTP para a API
React Router DOM	6.x	Roteamento client-side

Infraestrutura

Serviço	Plano	Uso
Supabase	Gratuito	PostgreSQL 14+ hospedado na nuvem
Upstash Redis	Gratuito	Cache de recomendações (recomendado no Windows)
Vercel (planejado)	Gratuito	Deploy do frontend React
Railway (planejado)	Pago	Deploy do backend Node.js

5. Estrutura do Banco de Dados

Prefixo qb_ evita conflito com outro projeto no mesmo Supabase. IDs são bigint GENERATED ALWAYS AS IDENTITY — FK UUID×bigint causava erro.

Tabela	Colunas principais	Papel
qb_clientes	id, nome_loja, cidade, uf	Barbearias clientes
qb_produtos	id, nome, categoria, preco, ativo	Catálogo de produtos
qb_vendas	id, cliente_id, data_venda, valor_total	Cabeçalho de transações
qb_itens_venda	id, venda_id, produto_id, quantidade, valor_unitario	Itens de cada venda (chave p/ algoritmo)
qb_regras_associacao	id, antecedente_id, consequente_id, confidence, support, lift	Cache persistente de regras

Join crítico — buscarVendas()

```
// qb_itens_venda JOIN qb_produtos via PostgREST
const { data } = await supabase
  .from('qb_itens_venda')
  .select('venda_id, quantidade, qb_produtos (id, nome, categoria)');

// Resultado agrupado por venda_id (feito em memória com vendasMap):
// [ { id: 1, itens: [{ nome_produto: 'Fox Barba', ... }] }, ... ]
```

RLS desabilitado: Row Level Security desabilitado nas tabelas qb_* via ALTER TABLE qb_* DISABLE ROW LEVEL SECURITY — padrão bloqueava reads com anon key retornando [] silenciosamente.

6. Algoritmo de Recomendação — Association Rules

As três métricas

Métrica	Fórmula	Significado
Support	$\text{freq}(A \cap B) / \text{total}$	Quão comum é o par A+B. Filtra pares raros.
Confidence	$\text{freq}(A \cap B) / \text{freq}(A)$	$P(B A)$: probabilidade de B dado que A foi comprado.
Lift	$\text{confidence}(A \rightarrow B) / P(B)$	Lift > 1 = correlação real; = 1 = independência.

Thresholds

Parâmetro	Valor	Arquivo
MIN_CONFIDENCE	0.30 (30%)	src/config/constants.js
MIN_SUPPORT	0.02 (2%)	src/config/constants.js
TOP_N	3 recomendações	src/config/constants.js

Fluxo do algoritmo

```
// Para cada par (A, B) onde A ≠ B:  
freq_A = vendas onde A aparece  
freq_AB = vendas onde A E B aparecem  
  
confidence = freq_AB / freq_A // P(B|A)  
support = freq_AB / total // freq. conjunta  
lift = confidence / P(B) // correlação real?  
  
// Aceita regra se confidence >= 0.30 E support >= 0.02  
// Ordena por confidence desc → retorna top 3
```

Exemplo real testado

```
GET /api/recomendacoes/Fox%20For%20Men%20Barba  
  
→ Espuma para Barba Premium (confiança: 43%, lift: 1.8)  
→ Tônico de Barba (confiança: 43%, lift: 2.1)
```


Complexidade: $O(n \times p^2)$ – ~200k ops com 500 transações e 20 produtos
Com cache Redis: <10ms (vs ~350ms sem cache)

7. API REST — Endpoints

Método	Endpoint	Descrição	Cache TTL
GET	/health	Health check do servidor	—
GET	/api/recomendacoes/:produtoid	Top 3 recomendações para um produto	1 hora
GET	/api/estatisticas	Métricas globais + top 5 regras	30 min

Resposta — /api/recomendacoes/:produtoid

```
{
  "produto": "Fox For Men Barba",
  "total_recomendacoes": 2,
  "recomendacoes": [
    {
      "nome": "Espuma para Barba Premium",
      "confianca": 0.43,
      "support": 0.06,
      "lift": 1.8,
      "descricao": "Clientes que compraram Fox For Men Barba tambem compraram..."
    }
  ],
  "source": "computado" // ou "cache"
}
```

Resposta — /api/estatisticas

```
{
  "total_vendas": 150,
  "total_produtos": 20,
  "total_regras": 47,
  "confianca_media": 0.4823,
  "support_medio": 0.0812,
  "regras_top5": [ { "antecedente": "...", "consequente": "...", ... } ],
  "source": "computado"
}
```

Códigos de erro

HTTP	Causa
400	produtoid vazio ou nenhuma venda no banco

8. Cache Redis

Estratégia cache-aside

- 1. Verifica Redis pela chave → se hit, retorna imediatamente
- 2. Se miss: busca Supabase → calcula algoritmo → armazena no Redis
- 3. Retorna resultado com campo source: "cache" ou "computado"

Degradação graceful

Se o Redis estiver offline, todas as funções do cache retornam silenciosamente (get → null, set → void) sem lançar erros. A API continua funcionando normalmente, apenas sem o benefício do cache.

~350ms

Latência sem cache (1ª chamada)

<10ms

Latência com cache ativo

35×

Redução de latência

Configuração Redis no Windows

```
# Opção recomendada: Upstash (cloud gratuito)
# 1. Criar conta em upstash.com
# 2. Criar database Redis
# 3. Copiar REDIS_URL para backend/.env:
REDIS_URL=redis://default:SENHA@endpoint.upstash.io:6379

# Alternativa: WSL2 local
wsl -e redis-server
```

9. Testes Unitários

16

Total de testes passando

10

Testes do algoritmo

6

Testes de cache

recomendacao.test.js — cobertura

Função	Casos testados
extrairProdutosUnicos()	Set com todos os produtos; Set vazio para array vazio
calculateAssociation()	Retorna array; filtro confidence; ordenação desc; dataset vazio; campos obrigatórios
obterRecomendacoes()	Limite topN; apenas produto correto; produto inexistente

cache.test.js — cobertura

Caso	Comportamento
isConnected()	Retorna boolean
get() chave inexistente	Retorna null
set() + get() com Redis ativo	Armazena e recupera (pulado se offline)
delete() com Redis ativo	Remove chave (pulado se offline)
set() sem Redis	Não lança erro (degradação graceful)
get() sem Redis	Retorna null silenciosamente

Executar testes

```
cd backend
npm run test:unit      # Apenas testes unitários (--forceExit)
npm run test:coverage  # Relatório de cobertura em /coverage/
```

10. Progresso por Semana

 Semana 1 Estrutura inicial	 Semana 2 Setup completo	 Semana 3 Algoritmo + testes	 Semana 4 Cache + Estatísticas
 Semana 5 Validação algoritmo	 Semana 6 Dashboard frontend	 Semana 7 Gráficos + visuais	 Semana 8 Testes completos
 Semana 9 TCC: Intro + Fund.	 Semana 10 TCC: Metodologia	 Semana 11 TCC: Resultados	 Semana 12 Apresentação

Semana 1 — Estrutura inicial

- Repositório GitHub criado
- Estrutura de pastas: backend/, frontend/, docs/
- Roteiros semanas 1-12, sumário executivo, rastreador de progresso
- Commit: 9f75431

Semana 2 — Setup completo do ambiente

- Backend: Express + dotenv + cors + nodemon + jest
- Supabase: 5 tabelas qb_* com bigint identity, RLS desabilitado
- Seed SQL com 20 produtos e 150+ vendas mock com itens por pedido
- Frontend: React 18 + Vite + Axios + React Router, proxy /api configurado
- Decisão chave: Supabase JS client em vez de Knex (sem DATABASE_URL disponível)

Semana 3 — Algoritmo de recomendação

- services/recomendacao.js: extrairProdutosUnicos(), calculateAssociation(), obterRecomendacoes()
- Endpoint GET /api/recomendacoes/:produtold funcional e testado com curl
- 10 testes unitários passando (Jest) cobrindo casos felizes e edge cases

Semana 4 — Cache Redis + Endpoint de Estatísticas

- services/cache.js: get/set/delete/flush com degradação graceful sem Redis
- Cache integrado no endpoint de recomendações (TTL 1h, log CACHE HIT/MISS)
- GET /api/estatisticas: total_vendas, produtos, regras, confiança_media, top5 regras
- 6 novos testes de cache — total: 16 testes passando
- Commits: de0ba8e (Semanas 2+3) e b8befe5 (Semana 4)

Progresso geral — Maio/2026

Backend (80%):



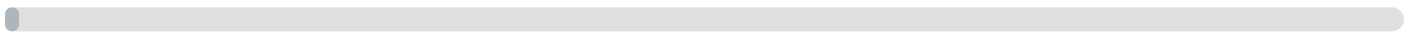
Frontend (15%):



Testes (60%):



TCC — texto acadêmico (0%):



11. Arquivos do Projeto

Arquivo	Status	Descrição
backend/.env	✓	Credenciais Supabase + config (não commitado, .gitignore)
backend/src/app.js	✓	Servidor Express, middlewares CORS/JSON, rotas
backend/src/config/constants.js	✓	MIN_CONFIDENCE, MIN_SUPPORT, TOP_N, CACHE_TTL
backend/src/database/connection.js	✓	Cliente Supabase JS (PostgREST)
backend/src/database/schema.sql	✓	5 tabelas qb_* com bigint identity e índices
backend/src/database/seed.sql	✓	Dados mock — rodar no SQL Editor do Supabase
backend/src/database/seed.js	⚠ NÃO USAR	Script Node com problema de cache do PostgREST
backend/src/models/index.js	✓	buscarVendas() com join, buscarProdutos(), buscarProduto()
backend/src/services/recomendacao.js	✓	Algoritmo Association Rules completo e comentado
backend/src/services/cache.js	✓	Camada Redis com degradação graceful
backend/src/routes/recomendacoes.js	✓	GET /api/recomendacoes/:produtoId com cache
backend/src/routes/estatisticas.js	✓	GET /api/estatisticas com métricas globais
backend/tests/unit/recomendacao.test.js	✓	10 testes do algoritmo (todos passando)
backend/tests/unit/cache.test.js	✓	6 testes de cache (degradação graceful inclusa)
frontend/src/main.jsx	✓	Entry point React com StrictMode
frontend/src/App.jsx	✓	BrowserRouter + Routes + placeholder Semana 2
frontend/src/services/api.js	✓	Cliente Axios centralizado com obterRecomendacoes()
frontend/src/styles/index.css	✓	Reset CSS básico

12. Decisões Técnicas e Armadilhas

Decisão / Armadilha	Causa	Solução aplicada
Prefixo qb_ nas tabelas	Supabase compartilhado com outro projeto (já tinha tabela "vendas")	Prefixo qb_ em todas as 5 tabelas
bigint em vez de UUID	FK UUID → bigint causava erro de tipo; Supabase cria bigint por padrão	Schema inteiro migrado para bigint GENERATED ALWAYS AS IDENTITY
Supabase JS client em vez de Knex	Knex precisa de DATABASE_URL com senha do postgres (não disponível no free tier)	@supabase/supabase-js com anon key via PostgREST
Seed via SQL Editor (não Node)	PostgREST tem cache de schema — script Node não via colunas novas imediatamente	seed.sql para rodar direto no Supabase SQL Editor
RLS bloqueava reads silenciosamente	Novas tabelas têm RLS ativo por padrão — reads retornavam [] sem erro	ALTER TABLE qb_* DISABLE ROW LEVEL SECURITY
Redis não disponível no Windows	Sem brew, sem apt — Redis não é nativo no Windows	Degradação graceful no cache.js + recomendação do Upstash (cloud gratuito)
CSV Vendas_2026.csv inútil para o algoritmo	Formato só tem totais por cliente, sem produtos por pedido	Dados mock realistas no seed.sql; trocar quando relatório detalhado disponível

13. Próximos Passos (Semanas 5–12)

Semana	Tema	Principais entregas
5	Validação do algoritmo	Train/test split, Precision, Recall, F1 reais; script de validação
6	Dashboard Frontend	BuscadorProduto, CardRecomendacao, tela de busca interativa
7	Gráficos	Chart.js: barras de confiança, scatter lift vs support, KPIs
8	Testes completos	Cobertura >80%, testes de integração API, testes React
9	TCC: Intro + Fundamentação	Capítulos 1 e 2, revisão bibliográfica (5+ artigos)
10	TCC: Metodologia + Implementação	Capítulos 3 e 4, diagramas UML, fluxograma do algoritmo
11	TCC: Resultados + Conclusão	Capítulos 5 e 6, análise das métricas, limitações e trabalhos futuros
12	Apresentação + Defesa	Slides, ensaio, ajustes finais do texto, submissão

Ações imediatas antes da Semana 5:

- 1. Configurar Upstash Redis (upstash.com) e atualizar REDIS_URL no backend/.env
- 2. Exportar relatório de vendas com itens por pedido do sistema da Quasar Barber
- 3. Substituir dados mock pelo dataset real no seed.sql